UNIVERSIDAD TÉCNICA DE AMBATO

Carrera de Ingeniería de Software

Prueba_Practica_Arboles_CPP_Java

Asignatura	Estructura de Datos
Nivel	Tercer nivel B
Lenguaje	C++, Java
Estudiante	Acurio Lopez Paul Medardo
Docente	Ing. José Caiza
Fecha	15 de mayo de 2026

Propósito del documento. Esta tarea detalla la construcción y evaluación de un Árbol Binario de Búsqueda (BST) aplicado a un Sistema Académico, integrando estructuras dinámicas no lineales, el uso de colas y recursividad vistas en clase para gestionar y analizar registros de estudiantes de forma interactiva.

1. Descripción general

Las estructuras de datos jerárquicas permiten organizar información de forma que las operaciones de búsqueda y recorrido sean eficientes. Este proyecto se enfoca en representar registros académicos mediante un Árbol Binario de Búsqueda en C++ y Java. El sistema desarrollado permite gestionar estudiantes, insertar nuevos nodos validando que no existan números de cédula duplicados, y analizar el rendimiento académico mediante recorridos de profundidad (DFS: Preorden, Inorden, Postorden) y de anchura (BFS). Además, incluye funciones de analítica como el cálculo de estudiantes aprobados/reprobados y la búsqueda de notas extremas.

2. Justificación académica

El proyecto es pertinente para la asignatura porque integra, en una sola aplicación, contenidos clave de Estructura de Datos: TDA, nodos enlazados, punteros (C++), referencias a objetos (Java), colas, recursividad y recorridos en árboles. En particular, el recorrido BFS se aplicó como una implementación directa de la cola trabajada en clase (queue en C++ y LinkedList actuando como Queue en Java), mientras que los algoritmos DFS y los cálculos de altura del árbol se vincularon con recursividad controlada.

3. Objetivo general

Desarrollar un sistema académico en C++ y Java que represente y gestione un árbol binario de búsqueda, aplicando operaciones de inserción estructurada, eliminación de nodos, recorridos (DFS y BFS), y cálculo de métricas académicas (notas y estados de aprobación) de forma interactiva para el usuario.

4. Objetivos específicos

1. Representar una jerarquía mediante una estructura de árbol binario usando un modelo de datos (payload) que contenga cédula, apellidos, nombres, nota final, carrera y nivel.
2. Implementar operaciones de agregación, búsqueda por clave (cédula) y eliminación considerando los tres casos lógicos de un BST.
3. Aplicar recorridos BFS y DFS para explorar la base de datos de estudiantes.
4. Aplicar la teoría de los recorridos a un caso de negocio real del proyecto final.
5. Evaluar el conocimiento del usuario mediante un cuestionario interactivo de recorridos.
6. Aplicar la teoría de los recorridos a un caso de negocio real (Módulos de un Sistema Web).
7. Analizar la complejidad temporal de las operaciones implementadas.

5. Explicación de la Estructura Utilizada

Para la representación del árbol y la gestión de estudiantes, se diseñó un programa iterativo con un menú principal de 14 opciones. La implementación técnica se fundamentó en los siguientes elementos:

- **Clase Nodo / Estructura:** Actúa como el bloque fundamental. Contiene un objeto o estructura `Estudiante` y dos referencias (punteros en la versión C++) hacia los nodos descendientes izquierdo y derecho.
- **Recursividad (DFS):** Los recorridos en profundidad (Preorden, Inorden y Postorden), así como el cálculo de la altura del árbol y la búsqueda de la nota mayor/menor, se implementaron mediante funciones recursivas. Esto minimiza el código y utiliza la pila de ejecución del sistema para navegar por las ramas.
- **Colas (BFS):** Para el recorrido por niveles (Búsqueda en Anchura), se implementó la interfaz Queue para procesar a los estudiantes nivel por nivel jerárquico.
- **Validación de Inserción:** Se implementó una función de comparación que impide el ingreso de estudiantes con cédulas duplicadas, manteniendo la integridad fundamental de un Árbol Binario de Búsqueda ordenado por número de identificación.

6. Definición de Conceptos Clave

Para comprender la funcionalidad del árbol implementado, se definen los siguientes elementos:

- **Raíz:** Es el nodo de origen (el primer estudiante ingresado) del cual se desprenden todos los demás. No posee un nodo superior.
- **Hoja:** Nodos en los extremos inferiores que no tienen hijos (ambas referencias apuntan a nulo).
- **Recorridos DFS:** Estrategias que exploran hasta la hoja más profunda antes de retroceder.
    - *Preorden:* Raíz → Izquierda → Derecha.
    - *Inorden:* Izquierda → Raíz → Derecha. En nuestro caso, imprime a los estudiantes ordenados numéricamente por cédula.
    - *Postorden:* Izquierda → Derecha → Raíz.
- **Recorrido BFS:** Estrategia que procesa los nodos nivel por nivel, de izquierda a derecha, utilizando una cola como estructura de apoyo.

7. Análisis de Complejidad Temporal

La implementación de esta estructura binaria presenta los siguientes rendimientos:

- **Recorridos y Métricas:** La complejidad es estrictamente de O(n) ya que, para imprimir los recorridos, filtrar aprobados/reprobados o buscar la nota más alta, el algoritmo debe visitar obligatoriamente todos y cada uno de los nodos del árbol exactamente una vez.
- **Inserción, Búsqueda y Eliminación:** En un árbol balanceado, la complejidad de buscar una cédula o insertar un nuevo estudiante es de O(log n). Si el usuario ingresa cédulas ordenadas secuencialmente (ej. 1, 2, 3...), el árbol puede degenerar en una lista enlazada, afectando el rendimiento al peor caso O(n).

8. Evidencia de Ejecución y Repositorio

El desarrollo colaborativo y los entregables se gestionan a través de control de versiones.

Enlace de GitHub: https://github.com/Paul-Acurio/Prueba_Practica_Arboles_CPP_Java.git

- **Codigo en C++**

main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include <queue>
#include <algorithm>
#include <limits> // Necesario para limpiar el buffer correctamente

using namespace std;

struct Estudiante {
    string cedula;
    string apellidos;
    string nombres;
    float notaFinal;
    string carrera;
    int nivel;
};

struct Nodo {
    Estudiante datos;
    Nodo* izq;
    Nodo* der;
};

class ArbolEstudiantes {
private:
    Nodo* raiz;

    Nodo* crearNodo(Estudiante est) {
        Nodo* nuevo = new Nodo();
        nuevo->datos = est;
        nuevo->izq = NULL;
        nuevo->der = NULL;
        return nuevo;
    }

    Nodo* insertar(Nodo* nodo, Estudiante est) {
        if (nodo == NULL) return crearNodo(est);
```

```cpp
        if (est.cedula < nodo->datos.cedula)
            nodo->izq = insertar(nodo->izq, est);
        else if (est.cedula > nodo->datos.cedula)
            nodo->der = insertar(nodo->der, est);
        else
            cout << "\n[!] Error: El estudiante con esta cédula ya existe.\n";
        return nodo;
    }

    Nodo* buscar(Nodo* nodo, string cedula) {
        if (nodo == NULL || nodo->datos.cedula == cedula) return nodo;
        if (nodo->datos.cedula < cedula) return buscar(nodo->der, cedula);
        return buscar(nodo->izq, cedula);
    }

    Nodo* encontrarMinimo(Nodo* nodo) {
        Nodo* actual = nodo;
        while (actual && actual->izq != NULL) actual = actual->izq;
        return actual;
    }

    Nodo* eliminar(Nodo* nodo, string cedula) {
        if (nodo == NULL) return nodo;
        if (cedula < nodo->datos.cedula)
            nodo->izq = eliminar(nodo->izq, cedula);
        else if (cedula > nodo->datos.cedula)
            nodo->der = eliminar(nodo->der, cedula);
        else {
            if (nodo->izq == NULL) {
                Nodo* temp = nodo->der;
                delete nodo;
                return temp;
            } else if (nodo->der == NULL) {
                Nodo* temp = nodo->izq;
```

```cpp
                delete nodo;
                return temp;
            }
            Nodo* temp = encontrarMinimo(nodo->der);
            nodo->datos = temp->datos;
            nodo->der = eliminar(nodo->der, temp->datos.cedula);
        }
        return nodo;
    }

    void inorden(Nodo* nodo) {
        if (nodo != NULL) {
            inorden(nodo->izq);
            cout << nodo->datos.cedula << " - " << nodo->datos.apellidos << " " << nodo->datos.nombres <<
            inorden(nodo->der);
        }
    }

    void preorden(Nodo* nodo) {
        if (nodo != NULL) {
            cout << nodo->datos.cedula << " - " << nodo->datos.apellidos << " " << nodo->datos.nombres <<
            preorden(nodo->izq);
            preorden(nodo->der);
        }
    }

    void postorden(Nodo* nodo) {
        if (nodo != NULL) {
            postorden(nodo->izq);
            postorden(nodo->der);
            cout << nodo->datos.cedula << " - " << nodo->datos.apellidos << " " << nodo->datos.nombres <<
```

```cpp
        }
    }

    int contar(Nodo* nodo) {
        if (nodo == NULL) return 0;
        return 1 + contar(nodo->izq) + contar(nodo->der);
    }

    int altura(Nodo* nodo) {
        if (nodo == NULL) return 0;
        return 1 + max(altura(nodo->izq), altura(nodo->der));
    }

    void notaMayor(Nodo* nodo, Nodo*& maxNodo) {
        if (nodo == NULL) return;
        if (maxNodo == NULL || nodo->datos.notaFinal > maxNodo->datos.notaFinal) maxNodo = nodo;
        notaMayor(nodo->izq, maxNodo);
        notaMayor(nodo->der, maxNodo);
    }

    void notaMenor(Nodo* nodo, Nodo*& minNodo) {
        if (nodo == NULL) return;
        if (minNodo == NULL || nodo->datos.notaFinal < minNodo->datos.notaFinal) minNodo = nodo;
        notaMenor(nodo->izq, minNodo);
        notaMenor(nodo->der, minNodo);
    }

    void mostrarEstado(Nodo* nodo, bool aprobados) {
        if (nodo != NULL) {
            mostrarEstado(nodo->izq, aprobados);
            bool esAprobado = nodo->datos.notaFinal >= 7.0;
            if ((aprobados && esAprobado) || (!aprobados && !esAprobado)) {
```

```cpp
                cout << nodo->datos.cedula << " - " << nodo->datos.apellidos << " " << nodo->datos.nombre
            }
            mostrarEstado(nodo->der, aprobados);
        }
    }

public:
    ArbolEstudiantes() { raiz = NULL; }

    void insertarEstudiante(Estudiante est) {
        raiz = insertar(raiz, est);
        cout << "[+] Estudiante insertado/procesado correctamente.\n";
    }

    void buscarEstudiante(string cedula) {
        Nodo* res = buscar(raiz, cedula);
        if (res) cout << "Encontrado: " << res->datos.apellidos << " " << res->datos.nombres << "\nCarrer
        else cout << "[-] Estudiante no encontrado.\n";
    }

    void eliminarEstudiante(string cedula) {
        if (buscar(raiz, cedula)) {
            raiz = eliminar(raiz, cedula);
            cout << "[+] Estudiante eliminado correctamente.\n";
        } else {
            cout << "[-] Estudiante no existe.\n";
        }
    }

    void recorridoInorden() { if(raiz==NULL) cout<<"Árbol vacío.\n"; else inorden(raiz); }
    void recorridoPreorden() { if(raiz==NULL) cout<<"Árbol vacío.\n"; else preorden(raiz); }
    void recorridoPostorden() { if(raiz==NULL) cout<<"Árbol vacío.\n"; else postorden(raiz); }
```

```cpp

    void recorridoPorNiveles() {
        if (raiz == NULL) { cout << "Árbol vacío.\n"; return; }
        queue<Nodo*> q;
        q.push(raiz);
        while (!q.empty()) {
            Nodo* actual = q.front();
            cout << actual->datos.cedula << " (" << actual->datos.apellidos << ") -> ";
            q.pop();
            if (actual->izq != NULL) q.push(actual->izq);
            if (actual->der != NULL) q.push(actual->der);
        }
        cout << "FIN\n";
    }

    void contarNodos() { cout << "Total de estudiantes en el sistema: " << contar(raiz) << "\n"; }
    void calcularAltura() { cout << "Altura máxima del árbol: " << altura(raiz) << "\n"; }

    void buscarNotaMayor() {
        Nodo* maxNodo = NULL;
        notaMayor(raiz, maxNodo);
        if (maxNodo) cout << "Mayor nota: " << maxNodo->datos.notaFinal << " (" << maxNodo->datos.apelli
        else cout << "El árbol está vacío.\n";
    }

    void buscarNotaMenor() {
        Nodo* minNodo = NULL;
        notaMenor(raiz, minNodo);
        if (minNodo) cout << "Menor nota: " << minNodo->datos.notaFinal << " (" << minNodo->datos.apelli
        else cout << "El árbol está vacío.\n";
    }

    void mostrarAprobados() { if(raiz==NULL) cout<<"Árbol vacío.\n"; else mostrarEstado(raiz, true); }
    void mostrarReprobados() { if(raiz==NULL) cout<<"Árbol vacío.\n"; else mostrarEstado(raiz, false); }
```

```cpp
};

// Función auxiliar para leer enteros de forma segura
int leerEntero(string mensaje) {
    int valor;
    cout << mensaje;
    while (!(cin >> valor)) {
        cout << "[!] Entrada inválida. Por favor, ingrese solo números enteros.\n" << mensaje;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    return valor;
}

// Función auxiliar para leer flotantes de forma segura
float leerFlotante(string mensaje) {
    float valor;
    cout << mensaje;
    while (!(cin >> valor)) {
        cout << "[!] Entrada inválida. Por favor, ingrese un número (ej. 8.5).\n" << mensaje;
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
    }
    return valor;
}

int main() {
    ArbolEstudiantes arbol;
    int opcion;

    do {
        cout << "\n=====================================\n";
        cout << "       SISTEMA ACADEMICO - UTA\n";
        cout << "=====================================\n";
```

```cpp
        cout << "1. Insertar estudiante\n";
        cout << "2. Buscar estudiante por cedula\n";
        cout << "3. Eliminar estudiante\n";
        cout << "4. Recorrido Inorden\n";
        cout << "5. Recorrido Preorden\n";
        cout << "6. Recorrido Postorden\n";
        cout << "7. Recorrido por niveles BFS\n";
        cout << "8. Contar estudiantes\n";
        cout << "9. Calcular altura del arbol\n";
        cout << "10. Mostrar estudiante con mayor nota\n";
        cout << "11. Mostrar estudiante con menor nota\n";
        cout << "12. Mostrar estudiantes aprobados\n";
        cout << "13. Mostrar estudiantes reprobados\n";
        cout << "14. Salir\n";
        cout << "=====================================\n";

        opcion = leerEntero("Seleccione una opcion: ");

        switch(opcion) {
            case 1: {
                cout << "\n--- 1. INSERTAR ESTUDIANTE ---\n";
                Estudiante est;
                cout << "Cedula: "; cin >> est.cedula;
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cout << "Apellidos: "; getline(cin, est.apellidos);
                cout << "Nombres: "; getline(cin, est.nombres);
                est.notaFinal = leerFlotante("Nota Final (ej. 8.5): ");
                cin.ignore(numeric_limits<streamsize>::max(), '\n');
                cout << "Carrera: "; getline(cin, est.carrera);
                est.nivel = leerEntero("Nivel (solo numero, ej. 3): ");
                arbol.insertarEstudiante(est);
                break;
            }
            case 2: {
```

```cpp
                cout << "\n--- 2. BUSCAR ESTUDIANTE ---\n";
                string ced; cout << "Ingrese la cedula a buscar: "; cin >> ced;
                arbol.buscarEstudiante(ced);
                break;
            }
            case 3: {
                cout << "\n--- 3. ELIMINAR ESTUDIANTE ---\n";
                string ced; cout << "Ingrese la cedula a eliminar: "; cin >> ced;
                arbol.eliminarEstudiante(ced);
                break;
            }
            case 4: cout << "\n--- 4. RECORRIDO INORDEN ---\n"; arbol.recorridoInorden(); break;
            case 5: cout << "\n--- 5. RECORRIDO PREORDEN ---\n"; arbol.recorridoPreorden(); break;
            case 6: cout << "\n--- 6. RECORRIDO POSTORDEN ---\n"; arbol.recorridoPostorden(); break;
            case 7: cout << "\n--- 7. RECORRIDO POR NIVELES (BFS) ---\n"; arbol.recorridoPorNiveles(); br
            case 8: cout << "\n--- 8. CONTAR ESTUDIANTES ---\n"; arbol.contarNodos(); break;
            case 9: cout << "\n--- 9. CALCULAR ALTURA ---\n"; arbol.calcularAltura(); break;
            case 10: cout << "\n--- 10. ESTUDIANTE CON MAYOR NOTA ---\n"; arbol.buscarNotaMayor(); break
            case 11: cout << "\n--- 11. ESTUDIANTE CON MENOR NOTA ---\n"; arbol.buscarNotaMenor(); break
            case 12: cout << "\n--- 12. ESTUDIANTES APROBADOS ---\n"; arbol.mostrarAprobados(); break;
            case 13: cout << "\n--- 13. ESTUDIANTES REPROBADOS ---\n"; arbol.mostrarReprobados(); break;
            case 14: cout << "\nSaliendo del sistema...\n"; break;
            default: cout << "\n[!] Opcion no valida. Intente de nuevo.\n"; break;
        }

    } while (opcion != 14);

    return 0;
}
```

- **Código en Java**

```java
import java.util.Scanner;
import java.util.LinkedList;
import java.util.Queue;

class Estudiante {  7 usages  Paul
    String cedula, apellidos, nombres, carrera;  15 usages
    float notaFinal;  13 usages
    int nivel;  2 usages

    public Estudiante(String cedula, String apellidos, String nombres, float notaFinal, String carrera, int nivel)
        this.cedula = cedula;
        this.apellidos = apellidos;
        this.nombres = nombres;
        this.notaFinal = notaFinal;
        this.carrera = carrera;
        this.nivel = nivel;
    }
}

class Nodo {  22 usages  Paul
    Estudiante datos;  39 usages
    Nodo izq, der;  21 usages

    public Nodo(Estudiante datos) {  1 usage  Paul
        this.datos = datos;
        this.izq = this.der = null;
    }
}

class ArbolEstudiantes {  2 usages  Paul
    private Nodo raiz;  23 usages

    public ArbolEstudiantes() { this.raiz = null; }  1 usage  Paul

    public void insertarEstudiante(Estudiante est) {  1 usage  Paul
        raiz = insertarRecursivo(raiz, est);
    }

    @Contract("null, _ -> new")
    private Nodo ! insertarRecursivo(Nodo raiz, Estudiante est) {  3 usages  Paul
        if (raiz == null) return new Nodo(est);
        if (est.cedula.compareTo(raiz.datos.cedula) < 0)
            raiz.izq = insertarRecursivo(raiz.izq, est);
        else if (est.cedula.compareTo(raiz.datos.cedula) > 0)
            raiz.der = insertarRecursivo(raiz.der, est);
        else
            System.out.println("[!] Error: El estudiante con esa cédula ya existe.");
        return raiz;
    }

    public void buscarEstudiante(String cedula) {  1 usage  Paul
        Nodo res = buscarRecursivo(raiz, cedula);
        if (res != null) {
            System.out.println("Encontrado: " + res.datos.apellidos + " " + res.datos.nombres);
            System.out.println("Carrera: " + res.datos.carrera + " | Nivel: " + res.datos.nivel);
            System.out.println("Nota Final: " + res.datos.notaFinal);
        } else {
            System.out.println("[-] Estudiante no encontrado.");
        }
    }
```

```java
@Contract("null, _ -> null")
private Nodo buscarRecursivo(Nodo raiz, String cedula) {  4 usages  Paul
    if (raiz == null || raiz.datos.cedula.equals(cedula)) return raiz;
    if (raiz.datos.cedula.compareTo(cedula) > 0) return buscarRecursivo(raiz.izq, cedula);
    return buscarRecursivo(raiz.der, cedula);
}

public void eliminarEstudiante(String cedula) {  1 usage  Paul
    if(buscarRecursivo(raiz, cedula) != null) {
        raiz = eliminarRecursivo(raiz, cedula);
        System.out.println("[+] Eliminado correctamente.");
    } else {
        System.out.println("[-] No existe el estudiante.");
    }
}

@Contract("null, _ -> null")
private Nodo eliminarRecursivo(Nodo raiz, String cedula) {  4 usages  Paul
    if (raiz == null) return raiz;
    if (cedula.compareTo(raiz.datos.cedula) < 0) raiz.izq = eliminarRecursivo(raiz.izq, cedula);
    else if (cedula.compareTo(raiz.datos.cedula) > 0) raiz.der = eliminarRecursivo(raiz.der, cedula);
    else {
        if (raiz.izq == null) return raiz.der;
        else if (raiz.der == null) return raiz.izq;

        raiz.datos = valorMinimo(raiz.der);
        raiz.der = eliminarRecursivo(raiz.der, raiz.datos.cedula);
    }
    return raiz;
```

```java
@Contract(mutates = "this")
private Estudiante valorMinimo(Nodo ! raiz) {  1 usage  Paul
    Estudiante minV = raiz.datos;
    while (raiz.izq != null) {
        minV = raiz.izq.datos;
        raiz = raiz.izq;
    }
    return minV;
}

public void recorridoInorden() { if(raiz==null) System.out.println("Árbol vacío."); else inorden(raiz); }  1 u
private void inorden(Nodo raiz) {  3 usages  Paul
    if (raiz != null) {
        inorden(raiz.izq);
        System.out.println(raiz.datos.cedula + " - " + raiz.datos.apellidos + " | Nota: " + raiz.datos.notaFi
        inorden(raiz.der);
    }
}

public void recorridoPreorden() { if(raiz==null) System.out.println("Árbol vacío."); else preorden(raiz); }
private void preorden(Nodo raiz) {  3 usages  Paul
    if (raiz != null) {
        System.out.println(raiz.datos.cedula + " - " + raiz.datos.apellidos + " | Nota: " + raiz.datos.notaFi
        preorden(raiz.izq);
        preorden(raiz.der);
    }
}
```

```java
    public void recorridoPostorden() { if(raiz==null) System.out.println("Árbol vacío."); else postorden(raiz);
    private void postorden(Nodo raiz) {  3 usages  Paul
        if (raiz != null) {
            postorden(raiz.izq);
            postorden(raiz.der);
            System.out.println(raiz.datos.cedula + " - " + raiz.datos.apellidos + " | Nota: " + raiz.datos.notaF
        }
    }

    public void recorridoPorNiveles() {  1 usage  Paul
        if (raiz == null) { System.out.println("Árbol vacío."); return; }
        Queue<Nodo> cola = new LinkedList<>();
        cola.add(raiz);
        while (!cola.isEmpty()) {
            Nodo actual = cola.poll();
            System.out.print(actual.datos.cedula + " (" + actual.datos.apellidos + ") -> ");
            if (actual.izq != null) cola.add(actual.izq);
            if (actual.der != null) cola.add(actual.der);
        }
        System.out.println("FIN");
    }

    public void contarNodos() { System.out.println("Total de estudiantes: " + contar(raiz)); }  1 usage  Paul
    private int contar(Nodo raiz) {  3 usages  Paul
        return (raiz == null) ? 0 : 1 + contar(raiz.izq) + contar(raiz.der);
    }

    public void calcularAltura() { System.out.println("Altura del árbol: " + altura(raiz)); }  1 usage  Paul
```

```java
    private int altura(Nodo raiz) {  3 usages  Paul
        return (raiz == null) ? 0 : 1 + Math.max(altura(raiz.izq), altura(raiz.der));
    }

    private Nodo maxNodo, minNodo;  7 usages

    public void buscarNotaMayor() {  1 usage  Paul
        maxNodo = null;
        mayorRecursivo(raiz);
        if (maxNodo != null) System.out.println("Mayor nota: " + maxNodo.datos.notaFinal + " (" + maxNodo.datos.
        else System.out.println("Árbol vacío.");
    }
    private void mayorRecursivo(Nodo nodo) {  3 usages  Paul
        if (nodo == null) return;
        if (maxNodo == null || nodo.datos.notaFinal > maxNodo.datos.notaFinal) maxNodo = nodo;
        mayorRecursivo(nodo.izq);
        mayorRecursivo(nodo.der);
    }

    public void buscarNotaMenor() {  1 usage  Paul
        minNodo = null;
        menorRecursivo(raiz);
        if (minNodo != null) System.out.println("Menor nota: " + minNodo.datos.notaFinal + " (" + minNodo.datos.
        else System.out.println("Árbol vacío.");
    }
    private void menorRecursivo(Nodo nodo) {  3 usages  Paul
        if (nodo == null) return;
        if (minNodo == null || nodo.datos.notaFinal < minNodo.datos.notaFinal) minNodo = nodo;
        menorRecursivo(nodo.izq);
```

```java
    }

    public void mostrarAprobados() {  1 usage  Paul
        if(raiz==null) System.out.println("Árbol vacío.");
        else { System.out.println("--- APROBADOS ---"); estadoRecursivo(raiz, aprobados: true); }
    }
    public void mostrarReprobados() {  1 usage  Paul
        if(raiz==null) System.out.println("Árbol vacío.");
        else { System.out.println("--- REPROBADOS ---"); estadoRecursivo(raiz, aprobados: false); }
    }
    private void estadoRecursivo(Nodo nodo, boolean aprobados) {  4 usages  Paul
        if (nodo != null) {
            estadoRecursivo(nodo.izq, aprobados);
            boolean paso = nodo.datos.notaFinal >= 7.0f;
            if (aprobados == paso) {
                System.out.println(nodo.datos.cedula + " - " + nodo.datos.apellidos + " | Nota: " + nodo.dato
            }
            estadoRecursivo(nodo.der, aprobados);
        }
    }
}

public class Main {  Paul
    public static void main(String[] args) {  Paul
        Scanner scanner = new Scanner(System.in);
        ArbolEstudiantes arbol = new ArbolEstudiantes();
        int opcion = 0;

        do {
```

```java
        do {
            System.out.println("\n========================================");
            System.out.println("       SISTEMA ACADÉMICO - UTA");
            System.out.println("========================================");
            System.out.println("1. Insertar estudiante\n2. Buscar estudiante\n3. Eliminar estudiante");
            System.out.println("4. Recorrido Inorden\n5. Recorrido Preorden\n6. Recorrido Postorden\n7. BFS (Po
            System.out.println("8. Contar estudiantes\n9. Altura del árbol\n10. Mayor nota\n11. Menor nota");
            System.out.println("12. Estudiantes aprobados\n13. Estudiantes reprobados\n14. Salir");
            System.out.println("========================================");
            System.out.print("Seleccione una opción: ");

            try {
                opcion = Integer.parseInt(scanner.nextLine());
            } catch (NumberFormatException e) {
                System.out.println("\n[!] Error: Debe ingresar un número entero válido.");
                continue; // Salta el resto del bucle y vuelve a mostrar el menú
            }

            switch (opcion) {
                case 1:
                    System.out.println("\n--- 1. INSERTAR ESTUDIANTE ---");
                    System.out.print("Cédula: "); String ced = scanner.nextLine();
                    System.out.print("Apellidos: "); String ape = scanner.nextLine();
                    System.out.print("Nombres: "); String nom = scanner.nextLine();

                    float nota = 0;
                    boolean notaValida = false;
                    while (!notaValida) {
                        System.out.print("Nota Final (ej. 8.5): ");
```

```java
                    try {
                        nota = Float.parseFloat(scanner.nextLine());
                        notaValida = true;
                    } catch (NumberFormatException e) {
                        System.out.println("[!] Error: Ingrese un formato numérico válido para la nota.");
                    }
                }

                System.out.print("Carrera: "); String car = scanner.nextLine();

                int niv = 0;
                boolean nivelValido = false;
                while (!nivelValido) {
                    System.out.print("Nivel (solo número entero, ej. 3): ");
                    try {
                        niv = Integer.parseInt(scanner.nextLine());
                        nivelValido = true;
                    } catch (NumberFormatException e) {
                        System.out.println("[!] Error: El nivel debe ser un número entero. No use letras.")
                    }
                }

                arbol.insertarEstudiante(new Estudiante(ced, ape, nom, nota, car, niv));
                System.out.println("[+] Estudiante registrado con éxito.");
                break;
            case 2:
                System.out.println("\n--- 2. BUSCAR ESTUDIANTE ---");
                System.out.print("Cédula a buscar: ");
                arbol.buscarEstudiante(scanner.nextLine());
                break;
            case 3:
                System.out.println("\n--- 3. ELIMINAR ESTUDIANTE ---");
                System.out.print("Cédula a eliminar: ");
                arbol.eliminarEstudiante(scanner.nextLine());
                break;
            case 4: System.out.println("\n--- 4. RECORRIDO INORDEN ---"); arbol.recorridoInorden(); break;
            case 5: System.out.println("\n--- 5. RECORRIDO PREORDEN ---"); arbol.recorridoPreorden(); break;
            case 6: System.out.println("\n--- 6. RECORRIDO POSTORDEN ---"); arbol.recorridoPostorden(); break;
            case 7: System.out.println("\n--- 7. RECORRIDO POR NIVELES (BFS) ---"); arbol.recorridoPorNiveles()
            case 8: System.out.println("\n--- 8. CONTAR ESTUDIANTES ---"); arbol.contarNodos(); break;
            case 9: System.out.println("\n--- 9. ALTURA DEL ÁRBOL ---"); arbol.calcularAltura(); break;
            case 10: System.out.println("\n--- 10. MAYOR NOTA ---"); arbol.buscarNotaMayor(); break;
            case 11: System.out.println("\n--- 11. MENOR NOTA ---"); arbol.buscarNotaMenor(); break;
            case 12: System.out.println("\n--- 12. ESTUDIANTES APROBADOS ---"); arbol.mostrarAprobados(); break
            case 13: System.out.println("\n--- 13. ESTUDIANTES REPROBADOS ---"); arbol.mostrarReprobados(); bre
            case 14: System.out.println("\nSaliendo del sistema..."); break;
            default: System.out.println("\n[!] Opción no válida. Intente de nuevo."); break;
        }
    } while (opcion != 14);

    scanner.close();
  }
}
```

- **Ejecución**

```
========================================
       SISTEMA ACADEMICO - UTA
========================================
1. Insertar estudiante
2. Buscar estudiante por cedula
3. Eliminar estudiante
4. Recorrido Inorden
5. Recorrido Preorden
6. Recorrido Postorden
7. Recorrido por niveles BFS
8. Contar estudiantes
9. Calcular altura del arbol
10. Mostrar estudiante con mayor nota
11. Mostrar estudiante con menor nota
12. Mostrar estudiantes aprobados
13. Mostrar estudiantes reprobados
14. Salir
========================================
Seleccione una opcion: 1
```

```
Seleccione una opcion: 1

--- 1. INSERTAR ESTUDIANTE ---
Cedula: 1850037878
Apellidos: acurio lopez
Nombres: paul medardo
Nota Final (ej. 8.5): 8
Carrera: software
Nivel (solo numero, ej. 3): 3
[+] Estudiante insertado/procesado correctamente.
```

```
========================================
Seleccione una opcion: 2

--- 2. BUSCAR ESTUDIANTE ---
Ingrese la cedula a buscar: 1850037878
Encontrado: acurio lopez paul medardo
Carrera: software
Nivel: 3
Nota: 8
```

```
Seleccione una opcion: 3

--- 3. ELIMINAR ESTUDIANTE ---
Ingrese la cedula a eliminar: 181215
[-] Estudiante no existe.
```

```
Seleccione una opcion: 4

--- 4. RECORRIDO INORDEN ---
1801802263 - acurio gamboa wagner medardo | Nota: 6
1850037878 - acurio lopez paul medardo | Nota: 8
```

```
==========================================
Seleccione una opcion: 5

--- 5. RECORRIDO PREORDEN ---
1850037878 - acurio lopez paul medardo | Nota: 8
1801802263 - acurio gamboa wagner medardo | Nota: 6
```

```
Seleccione una opcion: 6

--- 6. RECORRIDO POSTORDEN ---
1801802263 - acurio gamboa wagner medardo | Nota: 6
1850037878 - acurio lopez paul medardo | Nota: 8
```

```
Seleccione una opcion: 7

--- 7. RECORRIDO POR NIVELES (BFS) ---
1850037878 (acurio lopez) -> 1801802263 (acurio gamboa) -> FIN

==========================================
```

```
==========================================
Seleccione una opcion: 8

--- 8. CONTAR ESTUDIANTES ---
Total de estudiantes en el sistema: 2
```

```
==========================================
Seleccione una opcion: 9

--- 9. CALCULAR ALTURA ---
Altura m├íxima del ├árbol: 2
```

```
==========================================
Seleccione una opcion: 10

--- 10. ESTUDIANTE CON MAYOR NOTA ---
Mayor nota: 8 (acurio lopez paul medardo)
```

```
==========================================
Seleccione una opcion: 11

--- 11. ESTUDIANTE CON MENOR NOTA ---
Menor nota: 6 (acurio gamboa wagner medardo)
```

```
==========================================
Seleccione una opcion: 12

--- 12. ESTUDIANTES APROBADOS ---
1850037878 - acurio lopez paul medardo | Nota: 8
```

```
==========================================
Seleccione una opcion: 13

--- 13. ESTUDIANTES REPROBADOS ---
1801802263 - acurio gamboa wagner medardo | Nota: 6
```

9. Conclusiones

Cumplimiento de la Rúbrica: Se alcanzó el 100% de los resultados exigidos al implementar el sistema académico tanto en el paradigma estructurado con punteros (C++) como en el paradigma orientado a objetos (Java), abarcando todos los criterios de calificación.

Eficiencia de la Estructura: Se comprobó la flexibilidad de los árboles binarios. Al adaptar la estructura de nodos para contener un bloque de datos robusto (modelo Estudiante con notas y carreras), se demostró que un BST no solo sirve para ordenar números aislados, sino que es una base de datos en memoria sumamente rápida para operaciones CRUD.

Interactividad y Robustez: La inclusión de un menú con validación de tipos de datos (para evitar el colapso del buffer con Scanner y cin) convierte el script en una herramienta a prueba de errores humanos, garantizando la estabilidad que exige un entorno de producción o evaluación.